

# L43 — Collision Resolution: Chaining and Linear Probing

Unit: 3 | Chapter: Hashing | BT Level: BT5 | CO: CO2, CO3

---

## Learning Objectives

- Explain why collisions occur and why they're unavoidable (Birthday Paradox)
  - Implement separate chaining collision resolution
  - Implement open addressing with linear probing and quadratic probing
  - Compare collision resolution strategies
- 

## 1. The Birthday Paradox and Collision Inevitability

With  $m$  slots and  $n$  keys inserted, the probability of **at least one collision** is:

$$P(\text{collision}) \approx 1 - e^{-n^2/(2m)}$$

For  $m = 365$  (days) and  $n = 23$  people:  $P(\text{birthday collision}) \approx 50\%$ .

For a hash table with  $m = 10,000$  slots: collisions become likely after only  $\sim 125$  insertions. **Collisions are expected and must be handled properly.**

---

## 2. Strategy 1: Separate Chaining

**Each slot holds a linked list (chain) of all entries that hash to that slot.**

Table size = 7

Keys: 10, 22, 31, 4, 15, 28, 17, 88

$h(k) = k \% 7$ :

Slot 0: 28 → 35 → None

Slot 1: 15 → 22 → None

Slot 2: None

Slot 3: 10 → 31 → 17 → None

Slot 4: 4 → 88 → None

Slot 5: None

Slot 6: None

## Implementation with Chaining

```
class ChainedHashTable:
    def __init__(self, capacity=16):
        self.capacity = capacity
        self.size = 0
        self.table = [[] for _ in range(capacity)]
```

```

def _hash(self, key):
    return hash(key) % self.capacity

def insert(self, key, value):
    """O(1) average"""
    slot = self._hash(key)
    chain = self.table[slot]
    for i, (k, v) in enumerate(chain):
        if k == key:
            chain[i] = (key, value)
            return
    chain.append((key, value))
    self.size += 1

def search(self, key):
    """O(1 + chain_length) average → O(1) with low load factor"""
    slot = self._hash(key)
    for k, v in self.table[slot]:
        if k == key:
            return v
    return None

def delete(self, key):
    """O(1) average"""
    slot = self._hash(key)
    chain = self.table[slot]
    for i, (k, v) in enumerate(chain):
        if k == key:
            chain.pop(i)
            self.size -= 1
            return True
    return False

def load_factor(self):
    return self.size / self.capacity

```

## Chaining Analysis

### Load factor $\alpha$ Expected search time

|                |             |
|----------------|-------------|
| $\alpha = 0.5$ | 1.25 probes |
| $\alpha = 1.0$ | 1.5 probes  |
| $\alpha = 2.0$ | 2.0 probes  |

Formula: expected comparisons for unsuccessful search =  $1 + \alpha$ .  
 Chaining works well even for high load factors.

---

## 3. Strategy 2: Open Addressing

All entries stored directly in the table — no extra linked list nodes.

On collision, **probe** (search) for the next available slot.

**Load factor must stay < 1** (table can never be full).

### 3.1 Linear Probing

$$h(k, i) = (h(k) + i) \bmod m \quad \text{for } i = 0, 1, 2, \dots$$

```
class LinearProbingHashTable:
    DELETED = object()    # Sentinel for deleted slots

    def __init__(self, capacity=16):
        self.capacity = capacity
        self.size = 0
        self.keys = [None] * capacity
        self.values = [None] * capacity

    def _hash(self, key):
        return hash(key) % self.capacity

    def insert(self, key, value):
        """O(1) average, O(n) worst case"""
        if self.size >= self.capacity * 0.75:
            self._resize()

        slot = self._hash(key)
        i = 0
        while i < self.capacity:
            probe = (slot + i) % self.capacity
            if self.keys[probe] is None or self.keys[probe] is self.DELETED:
                self.keys[probe] = key
                self.values[probe] = value
                self.size += 1
                return
            elif self.keys[probe] == key:
                self.values[probe] = value    # Update
                return
            i += 1
        raise OverflowError("Hash table is full")

    def search(self, key):
        """O(1) average"""
        slot = self._hash(key)
        i = 0
        while i < self.capacity:
            probe = (slot + i) % self.capacity
            if self.keys[probe] is None:
                return None    # Empty slot → key not present
            if self.keys[probe] == key:
                return self.values[probe]
            i += 1
        return None
```

```

def delete(self, key):
    """Mark as DELETED (lazy deletion). O(1) average"""
    slot = self._hash(key)
    i = 0
    while i < self.capacity:
        probe = (slot + i) % self.capacity
        if self.keys[probe] is None:
            return False
        if self.keys[probe] == key:
            self.keys[probe] = self.DELETED
            self.values[probe] = None
            self.size -= 1
            return True
        i += 1
    return False

def _resize(self):
    old_keys = self.keys[:]
    old_values = self.values[:]
    self.capacity *= 2
    self.size = 0
    self.keys = [None] * self.capacity
    self.values = [None] * self.capacity
    for k, v in zip(old_keys, old_values):
        if k is not None and k is not self.DELETED:
            self.insert(k, v)

```

---

## Linear Probing Trace

**Table size  $m=7$ , insert: 10, 22, 31, 4, 15, 28, 17**

$h(k) = k \% 7$ :

| Key | $h(k)$ | Probe slots                            | Final slot |
|-----|--------|----------------------------------------|------------|
| 10  | 3      | 3 (empty)                              | 3          |
| 22  | 1      | 1 (empty)                              | 1          |
| 31  | 3      | 3 (10!) → 4 (empty)                    | 4          |
| 4   | 4      | 4 (31!) → 5 (empty)                    | 5          |
| 15  | 1      | 1 (22!) → 2 (empty)                    | 2          |
| 28  | 0      | 0 (empty)                              | 0          |
| 17  | 3      | 3 (10!) → 4 (31!) → 5 (4!) → 6 (empty) | 6          |

Table: [28, 22, 15, 10, 31, 4, 17]

**Primary clustering:** consecutive filled slots slow down future probes (slots 1–6 are filled!).

---

## 3.2 Quadratic Probing

$$h(k, i) = (h(k) + c_1 \cdot i + c_2 \cdot i^2) \bmod m$$

- Reduces **primary clustering** but can cause **secondary clustering**.

```
def quadratic_probe(slot, i, capacity, c1=1, c2=1):
    return (slot + c1 * i + c2 * i * i) % capacity
```

### 3.3 Double Hashing

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

- Two independent hash functions
- **No clustering** — best distribution among open addressing methods

```
def double_hash(key, i, m):
    h1 = key % m
    h2 = 1 + (key % (m - 1)) # Must be non-zero and coprime to m
    return (h1 + i * h2) % m
```

## 4. Comparison

| Feature                      | Separate Chaining      | Linear Probing     | Double Hashing |
|------------------------------|------------------------|--------------------|----------------|
| Memory                       | Extra node overhead    | Compact            | Compact        |
| Load factor                  | Any $\alpha$           | $\alpha < 1$       | $\alpha < 1$   |
| Cache performance            | Poor (pointer chasing) | Excellent          | Good           |
| Clustering                   | None                   | Primary clustering | None           |
| Deletion                     | Simple                 | Needs sentinel     | Needs sentinel |
| Performance at high $\alpha$ | Degrades gradually     | Degrades quickly   | Better         |

## 5. Complexity Summary

| Operation                         | Average           | Worst  |
|-----------------------------------|-------------------|--------|
| Chaining insert/search            | $O(1 + \alpha)$   | $O(n)$ |
| Linear probing ( $\alpha < 0.5$ ) | $O(1)$            | $O(n)$ |
| Linear probing ( $\alpha = 0.9$ ) | $\sim 10$ probes  | $O(n)$ |
| Double hashing                    | $O(1/(1-\alpha))$ | $O(n)$ |

## Summary

- Collisions are unavoidable — two strategies: **chaining** and **open addressing**.
- **Chaining**: Each slot is a list; simple, handles high load factors, but slower (pointer chasing).
- **Linear probing**: Probe + 1, + 2, + 3... — fast due to cache, but primary clustering.
- **Double hashing**: Two hash functions eliminate clustering but more complex.
- Keep load factor below 0.75 (chaining) or 0.5 (open addressing) for good performance.

## References

- Cormen et al., *Introduction to Algorithms*, Ch. 11.2–11.4 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 3.7 (Springer, 2020)